I2C MPR121QR2 Capacitive Touch Sensor

DEVLAB FACTOR

Mfr. Part #: UE0116

Product Reference Manual (V1.0)

Description

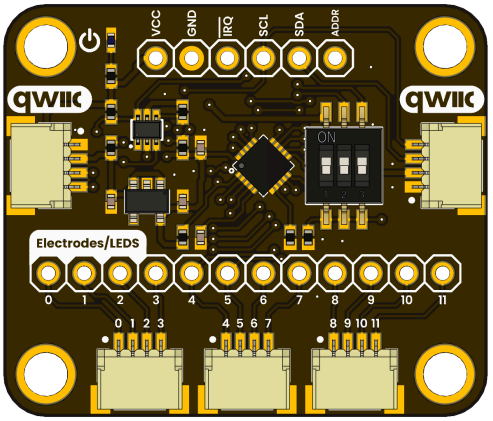
The MPR121 Development Module is a capacitive touch interface board based on the NXP MPR121 controller, providing 12 independent capacitive touch inputs over an I²C interface.

It is designed for rapid prototyping and product integration, combining flexible connector options, configurable I²C addressing via onboard DIP switches, and interrupt support for low-latency touch detection.

The module supports direct connection of electrodes or LEDs through dedicated JST connectors and 2.54 mm headers, making it suitable for both experimental setups and embedded applications where sealed or contactless user input is required.

DevLab format compatibility.

Simplicity and compatibility are the core principles of the **DevLab form factor**. This standard defines a **compact and communication-optimized board layout**, ensuring straightforward connection and interoperability among DevLab modules.



Key Features

- **12 independent capacitive touch inputs** (E0–E11)
- **I²C communication interface** with selectable addresses (0x5A–0x5D)
- **Three JST 1.0 mm connectors for touch electrodes or LEDs**
  Note: these connectors are used only for electrode/LED connections and are not I²C interfaces
- **Two I²C connectors compatible with DevLab / Qwiic / PiicoDev ecosystems**
- **Onboard DIP switch** for I²C address selection using ADDR pin logic
- **IRQ interrupt output** for efficient event-driven touch detection
- Compact PCB with mounting holes for mechanical integration

Hardware Features

- **Controller:** MPR121 capacitive touch sensor controller
- **Touch Inputs:** 12 electrodes (E0–E11)
- **I²C Address Selection:** Configurable via 3-position DIP switch (only one switch active at a time)
- **Connectors:**
  - 2 × JST 1.0 mm I²C connectors (Qwiic / DevLab / PiicoDev compatible)
  - 3 × JST 1.0 mm connectors for touch electrodes / LEDs
  - 2.54 mm header pads for electrodes and power signals
- **Interrupt:** Active-low IRQ pin
- **Power Supply:** 3.3 V logic and supply domain
- **Mounting:** Four corner mounting holes

Software Support

Arduino IDE

- Compatible with popular MPR121 Arduino libraries
- Supports polling and interrupt-driven modes

MicroPython

- I²C access supported on ESP32, RP2040, and similar platforms

Bare-metal / RTOS

- Simple I²C register interface suitable for STM32, NXP, GD32, ESP-IDF, and others

Applications

- Touch-sensitive user interfaces and control panels
- Musical instruments (keyboards, MIDI controllers, pads)
- Interactive art and exhibition installations
- Robotics and wearable electronics
- Industrial or sealed controls where mechanical buttons are not suitable

CONTENTS

Description 1
DevLab format compatibility. 1
Key Features 1
Hardware Features 2
Software Support 2
Applications 2
1. The Board 4
1.1 Accessories 4
2 Ratings 4
2.1 Recommended Operating Conditions 4
3. Functional Overview 5
I²C Interface and Addressing 5
Interrupt Output (IRQ) 6
3.2 Board Topology 8
4 Connectors & Pinouts 9
4.1 General Pinout 9
4.2 Pinout General Description 10
5 Board Operation 11
6. Mechanical Information 12
7 Company Information 13
8 Reference Documentation 13

1. The Board

1.1 Accessories

2 Ratings

2.1 Recommended Operating Conditions

Parameter	Description	Min	Typ	Max	Unit
Vcc	Input voltage to power on the module	3.3	-	5	V
Vih	High-level input voltage for I2C	0.7xVin	-	-	V
Vil	Low-level input voltage for I2C	-	-	0.3xVin	V
Vol	Low-level output voltage for SDA and IRQ	-	-	0.5V	V
Vohgpio	Output High Voltage ELE4 - ELE11 (GPIO mode) V_{DD}=3.3v | I_{OHGPIO}= -10 mA	VDD-0.5			V
Volgpio	Output Low Voltage ELE4 - ELE11 (GPIO mode) $I_{OLGPIOD}$ = 1 mA			0.5	V
Icc	Average Supply Current (Run1 Mode @ 1 ms sample period)	-	-	393	uA
fscl	Serial Clock Frequency	-	-	400	kHz

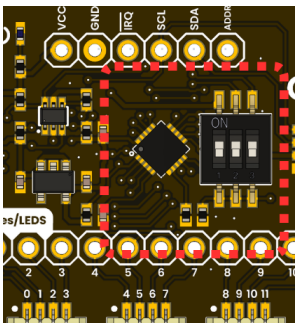
3. Functional Overview

The MPR121 Development Module is a capacitive touch interface board based on the MPR121 controller, providing up to 12 independent capacitive touch channels (E0–E11). The module detects changes in capacitance on each electrode and reports touch and release states digitally through an I²C interface.

The controller operates with 3.3 V recommended for optimal performance and signal stability. All digital signals are referenced to the same voltage domain, ensuring direct compatibility with 3.3 V microcontrollers and embedded systems.

I²C Interface and Addressing

Communication with the module is performed via I²C using the SDA and SCL signals. The I²C address is selectable through an onboard 3-position DIP switch, allowing multiple modules to share the same I²C bus.



Address Selection

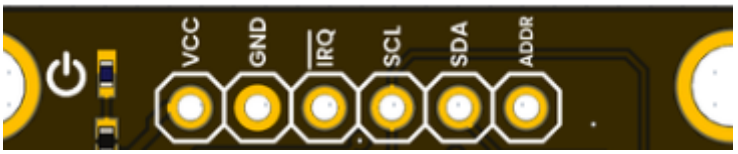
The I²C address is determined by the logic level on the **ADDR pin**:

ADDR Connection	Address
GND (VSS)	0x5A
VDD	0x5B
SDA	0x5C
SCL	0x5D

Important: Only one DIP switch must be active at a time. Enabling multiple switches will electrically connect different nodes (VDD, GND, SDA, SCL) through the ADDR pin, which may cause short circuits, I²C bus contention, and undefined device behavior.

Interrupt Output (IRQ)

An active-low IRQ pin is provided to signal touch state changes. When enabled, the interrupt line allows the host system to detect events without continuous I²C polling, improving responsiveness and reducing bus traffic.

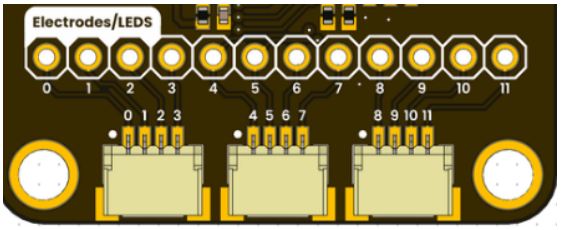


Touch Inputs and Connectors

The module exposes all 12 touch electrodes through multiple connector formats:

- JST connectors dedicated to touch electrodes or LEDs
  These connectors are not I²C interfaces and are intended exclusively for capacitive sensing or indicator connections.Two JST 1.0 mm I²C connectors compatible with DevLab, Qwiic, and PiicoDev ecosystems
- 2.54 mm header pads for direct wiring or prototyping

This combination allows the same module to be used in development, testing, and final product integration without hardware changes.



Pin Function Classification (Input / Output)

Pin Name	Type	Direction	Description
VDD	Power	Input	Supply voltage input
VSS (GND)	Power	—	Ground reference
VREG	Power	Output	Internal regulator output (requires 0.1 µF to GND, no external load)
SDA	Digital	I/O (Open-Drain)	I²C data line (bidirectional)
SCL	Digital	Input	I²C clock input
IRQ	Digital	Output (Open-Drain)	Interrupt output (active-low)

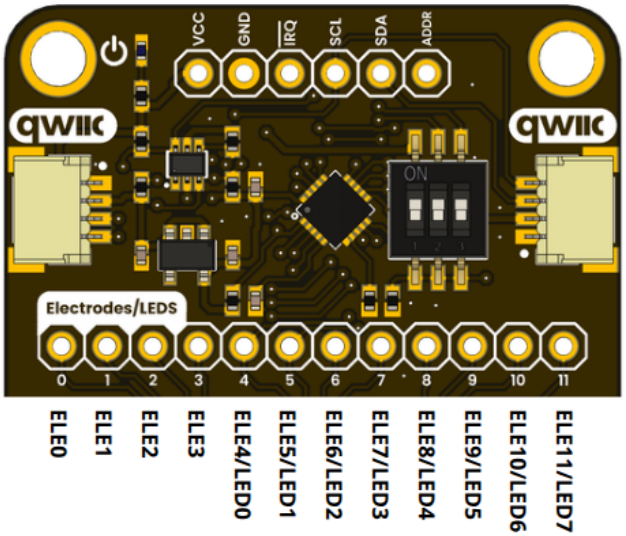
ADDR	Digital	Input	I²C address selection pin
REXT	Analog	Input	Reference resistor connection (75 kΩ to GND)

Electrode Pins (Multiplexed Function)

Los pines **ELE0 – ELE11** tienen doble funcionalidad:

- Entrada capacitiva (Touch)
- Salida digital (LED driver)

Pin	Mode	Direction	Description
ELE0 – ELE3	Touch / GPIO	Input / Output	Capacitive input or digital output
ELE4 – ELE11	Touch / LED	Input / Output	Capacitive input or LED driver output



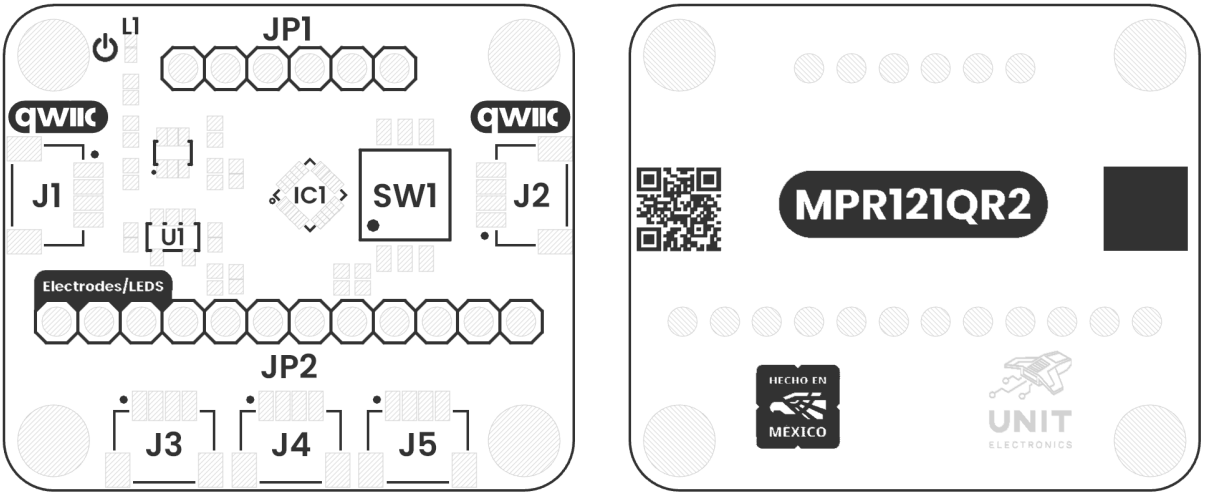
Functional Modes

Mode	Behavior
Touch Mode	Pins act as **capacitive inputs**
LED Mode	Pins act as **digital outputs** (LED drivers).

Important Constraints

- *Pins cannot operate simultaneously as touch inputs and outputs*
- *Mode selection is configured through internal registers*
- *Enabling LED mode disables capacitive sensing on those electrodes*
- *Internal multiplexing determines the active function*

3.2 Board Topology



Top Side **Bottom Side**

Views of Board Topology

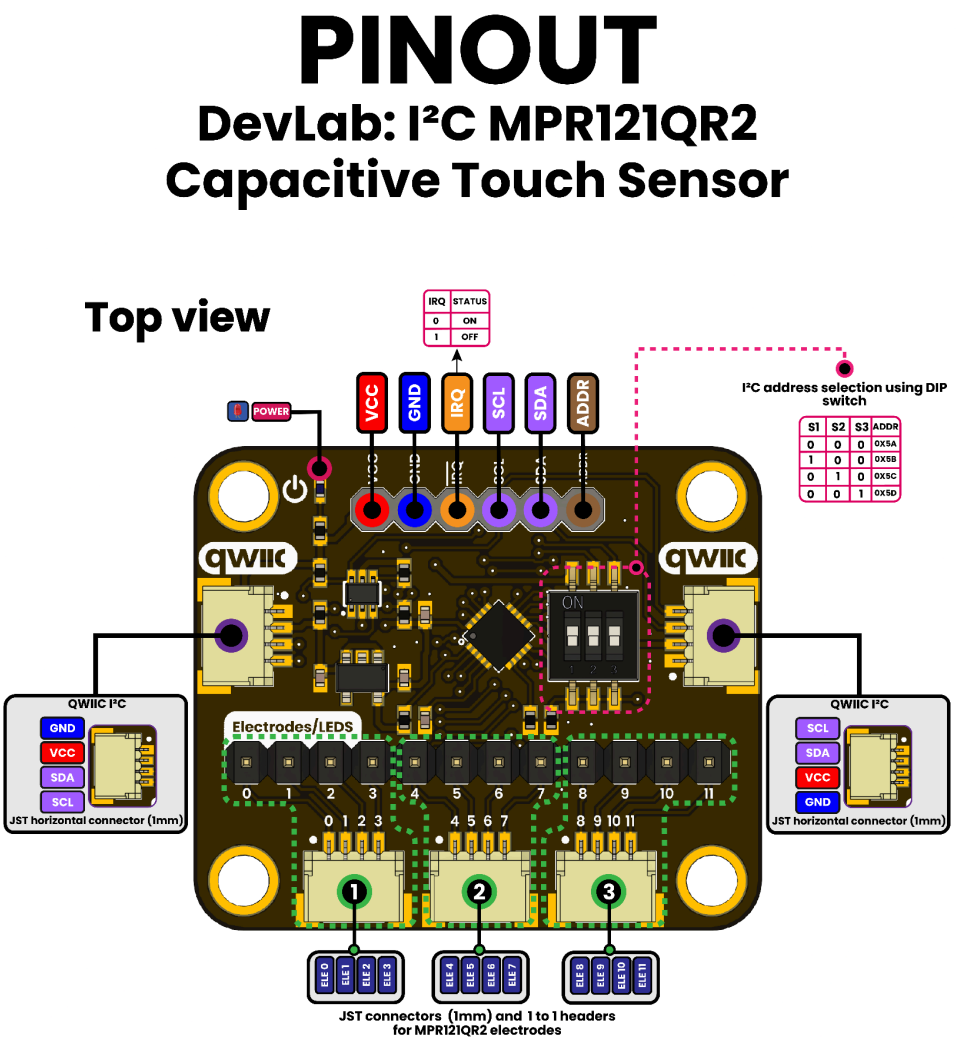
Views of Module Topology

Table 3.2.1 - Components Overview

Ref.	Description
C1	MPR121QR2
L1	Power On LED
U1	AP2112K 3.3V Regulator
JP1	2.54 mm Pin Headers for Control Signals
JP2	2.54 mm Pin Headers for Electrodes or LEDs
J1-J2	QWIIC Connector for I2C (JST 4-Pin 1mm)
J3-J5	JST 4-Pin 1mm Connector for Electrodes or LEDs
SW1	Dip Switch for Address Selection

4 Connectors & Pinouts

4.1 General Pinout



DevLab Module *General Pinout*

Important: Only one DIP switch must be active at a time. Enabling multiple switches will electrically connect different nodes (VDD, GND, SDA, SCL) through the ADDR pin, which may cause short circuits, I²C bus contention, and undefined device behavior.

4.2 Pinout General Description

Pin Label	Function	Notes
+3V3	Power Supply	3.3V Power Supply
GND	Ground	Common ground reference
SCL	I2C SCL	Serial Clock Line
SDA	I2C SDA	Serial Data Line

5 Board Operation

5.1 Getting Started

This section describes how to interface the **MPR121 capacitive touch controller** with the **PULSAR C6 (ESP32-C6)** using the Arduino IDE and verify touch input functionality.

5.1 Hardware Requirements

- **PULSAR C6 (ESP32-C6)**
- MPR121 Touch Module
- QWIIC / JST 1.0 mm cable or jumper wires
- Optional: external electrodes (pads, wires, conductive surfaces)

5.2 Electrical Connections (PULSAR C6)

Default DevLab / QWIIC Mapping

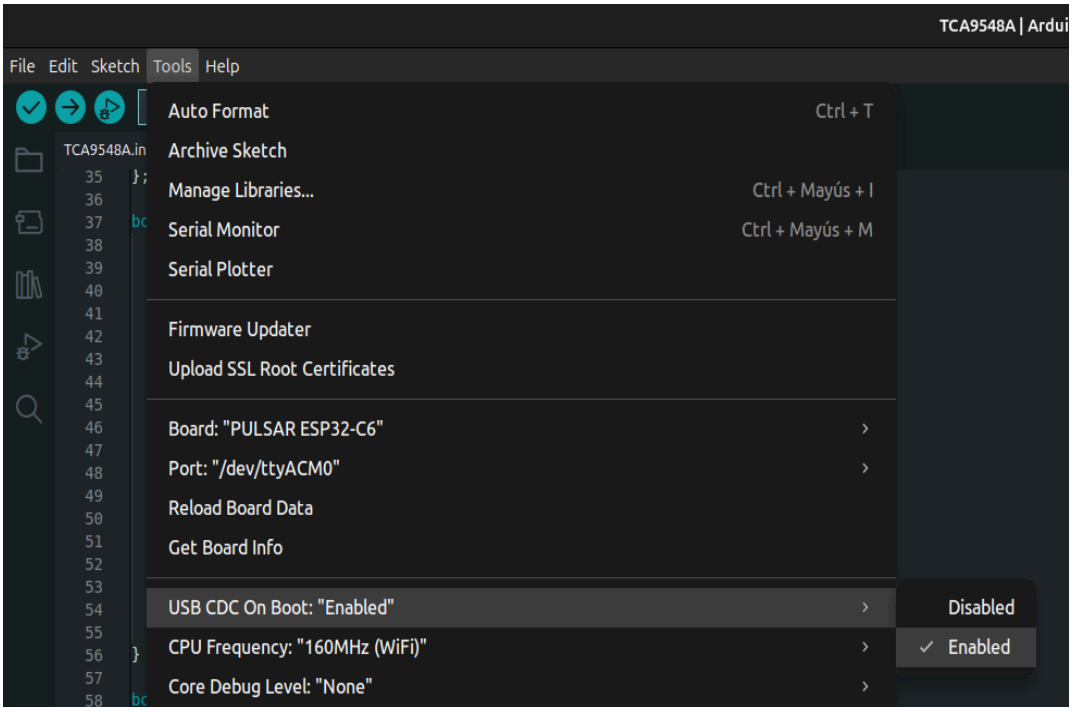
MPR121 Pin	PULSAR C6 Pin	Description
VDD	3.3V	Power supply
GND	GND	Ground
SDA	GPIO6	I²C Data
SCL	GPIO7	I²C Clock
IRQ (optional)	Any GPIO	Interrupt (optional)
ADDR	GND	Default I²C Address = **0x5A**

Important Electrical Notes

- *The PULSAR C6 operates at **3.3 V logic***
- *Ensure: Pull-ups on SDA/SCL (**4.7 kΩ typical**)*

5.3 Arduino IDE Configuration

1. Open **Arduino IDE**
2. Install ESP32 core:
   - Go to **Boards Manager**
   - Install: **ESP32 by Espressif Systems/UNIT Electronics ESP32**
3. Select board:
   - ESP32C6 Dev Module/Pulsar C6
4. Select correct COM port, For the Arduino example, enable USB CDC On Boot to use the serial monitor:



5.4 I²C Configuration (PULSAR C6)

The PULSAR C6 does not use fixed I²C pins, so they must be explicitly defined:

```
static const uint8_t SDA_PIN = 6;
static const uint8_t SCL_PIN = 7;
```

Initialize I²C:

```
Wire.begin(SDA_PIN, SCL_PIN);
```

5.5 Example: Touch Input Test

This example:

- Enables all 12 electrodes (E0–E11)
- Detects touch and release events
- Outputs results via Serial Monitor

Key Configuration Steps

1. Enter Stop Mode

```
writeReg(0x5E, 0x00);
```

2. Configure Filters

Registers:

```
0x2B – 0x32
```

3. Set Thresholds

```
TOUCH_THRESHOLD = 0x20
RELEASE_THRESHOLD = 0x10
```

4. Enable Electrodes

```
writeReg(0x5E, 0x0C); // 12 electrodes
```

5.6 Upload Procedure

1. Connect **PULSAR C6 via USB**
2. Upload the sketch
3. Open **Serial Monitor**
4. Set baud rate:

115200

5.7 Expected Output

When touching electrodes:

```
Basic input-only test ready
        Touch electrodes E0..E11 to see events

        TOUCH E0
        RELEASE E0
        TOUCH E5
        RELEASE E5
```

5.8 Troubleshooting (ESP32-C6 Specific)

Issue	Cause	Solution
No I²C response	Wrong pins	Verify GPIO6 / GPIO7
No device found	Wrong address	Confirm ADDR = GND (0x5A)

Random triggers	Noise	Reduce wire length
No serial output	USB CDC issue	Enable USB CDC in board settings
Crash / freeze	I²C init issue	Call Wire.begin() before config

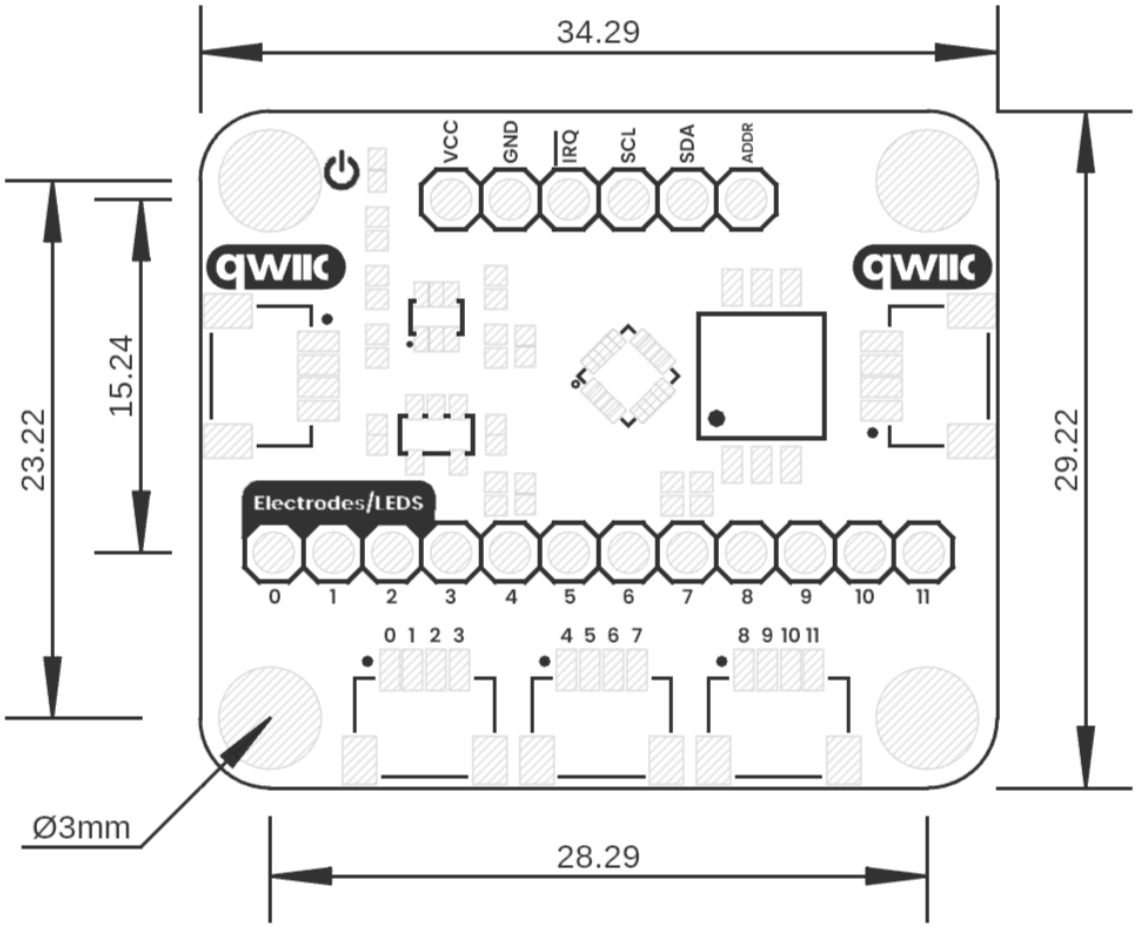
5.9 Example code

```
#include <Wire.h>

static const uint8_t SDA_PIN = 6;
static const uint8_t SCL_PIN = 7;
static const uint8_t MPR_ADDR = 0x5A;

static const uint8_t REG_TOUCH_STATUS_L = 0x00;
static const uint8_t REG_ELECTRODE_CONFIG = 0x5E;

static const uint8_t TOUCH_THRESHOLD = 0x20;
static const uint8_t RELEASE_THRESHOLD = 0x10;

static const uint8_t NUM_ELECTRODES = 12;

uint16_t lastTouchBits = 0;

void writeReg(uint8_t reg, uint8_t value) {
 Wire.beginTransmission(MPR_ADDR);
 Wire.write(reg);
 Wire.write(value);
 Wire.endTransmission();
 delayMicroseconds(100);
}

uint16_t readTouchStatus() {
 Wire.beginTransmission(MPR_ADDR);
 Wire.write(REG_TOUCH_STATUS_L);
 Wire.endTransmission(false);

 Wire.requestFrom(MPR_ADDR, (uint8_t)2);
 if (Wire.available() < 2) {
   return 0;
 }

 uint8_t lsb = Wire.read();
 uint8_t msb = Wire.read();
 return (uint16_t)lsb | ((uint16_t)msb << 8);
}

void printTouchEdges(uint16_t touchBits) {
```

```
 for (uint8_t i = 0; i < NUM_ELECTRODES; ++i) {
   uint16_t bit = (uint16_t)(1u << i);
   bool prev = (lastTouchBits & bit) != 0;
   bool curr = (touchBits & bit) != 0;

   if (!prev && curr) {
     Serial.print("TOUCH E");
     Serial.println(i);
   }

   if (prev && !curr) {
     Serial.print("RELEASE E");
     Serial.println(i);
   }
 }

 lastTouchBits = touchBits;
}

void initTouchOnly() {
 // 1) STOP mode
 writeReg(REG_ELECTRODE_CONFIG, 0x00);
 delay(10);

 // 2) Baseline filter configuration
 writeReg(0x2B, 0x01);
 writeReg(0x2C, 0x01);
 writeReg(0x2D, 0x00);
 writeReg(0x2E, 0x00);
 writeReg(0x2F, 0x01);
 writeReg(0x30, 0x01);
 writeReg(0x31, 0xFF);
 writeReg(0x32, 0x02);

 // 3) Touch / release thresholds for E0..E11
 for (uint8_t i = 0; i < NUM_ELECTRODES; ++i) {
   writeReg((uint8_t)(0x41 + (i * 2)), TOUCH_THRESHOLD);
   writeReg((uint8_t)(0x42 + (i * 2)), RELEASE_THRESHOLD);
 }

 // 4) RUN mode with all 12 electrodes enabled (0x0C = 12)
 writeReg(REG_ELECTRODE_CONFIG, 0x0C);
```

```
 delay(20);
}

void setup() {
 Serial.begin(115200);
 delay(200);

 Wire.begin(SDA_PIN, SCL_PIN);
 initTouchOnly();

 Serial.println("Basic input-only test ready");
 Serial.println("Touch electrodes E0..E11 to see events");
}

void loop() {
 uint16_t touchStatus = readTouchStatus();
 uint16_t touchBits = (uint16_t)(touchStatus & 0x0FFF);

 printTouchEdges(touchBits);

 delay(10);
}
```

6. Mechanical Information



Board Dimensions in Millimeters

Mechanical dimensions in millimeters

7 Company Information

Company name	UNIT Electronics
Company website	https://uelectronics.com/
Company Address	Salvador 19, Cuauhtémoc, 06000 Mexico City, CDMX

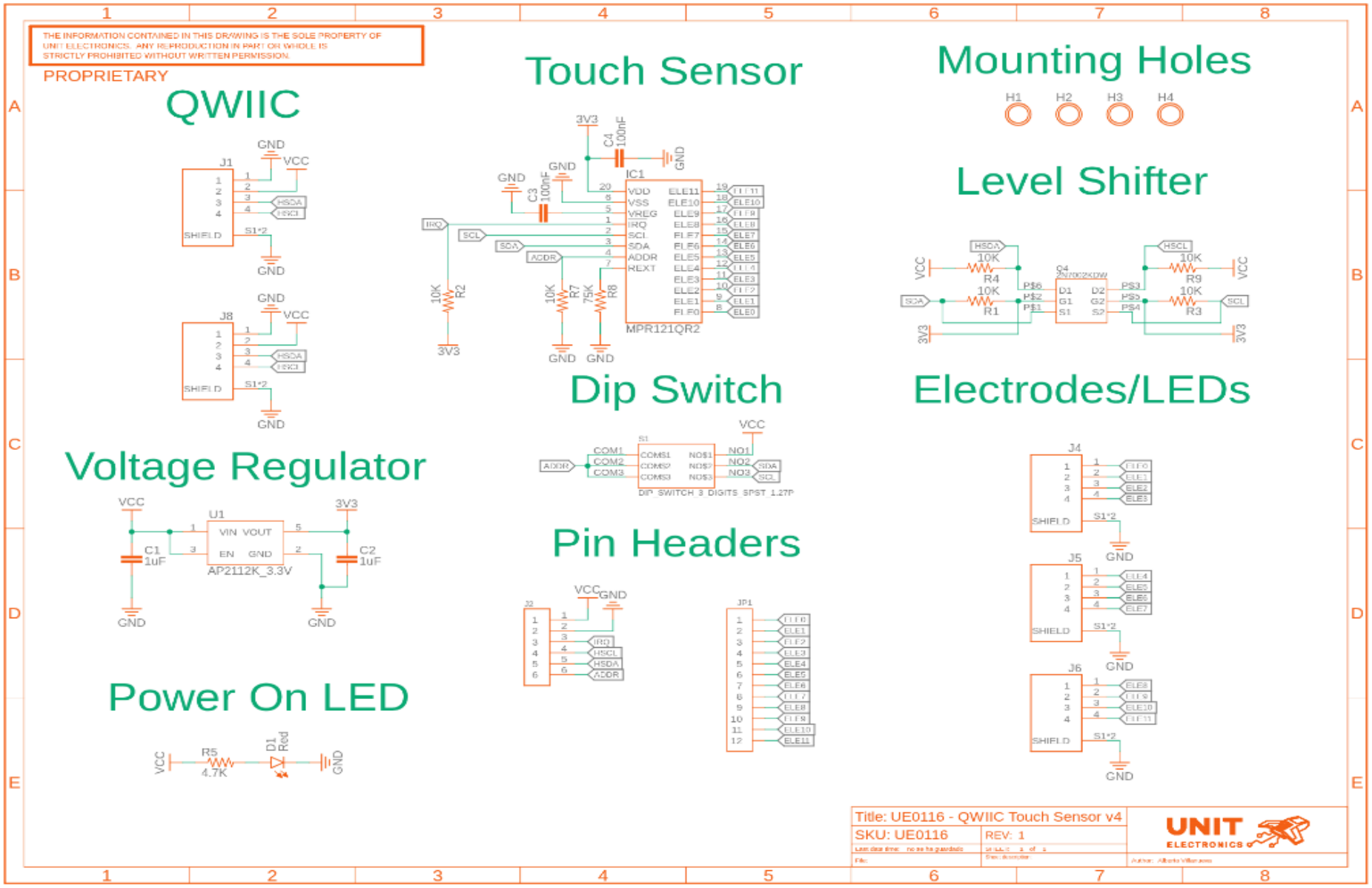
8 Reference Documentation

Ref	**Link**
Documentation	https://github.com/UNIT-Electronics-MX/devlab_i2c_mpr121qr2_capacitive_touch_sensor
Getting Started	https://wiki.uelectronics.com/wiki/devlab_i2c_mpr121qr2_capacitive_touch_sensor
Thonny IDE	https://thonny.org/
Arduino IDE	https://www.arduino.cc/en/software
Visual Studio Code	https://code.visualstudio.com/download

9 Appendix

9.1 Schematic

(*https://github.com/UNIT-Electronics-MX/unit_devlab_i2c_mpr121qr2_capacitive_touch_sensor/blob/main/hardware/unit_sch_v_1_0_0_ue0116_i2c_mpr121qr2_capacitive_touch_sensor.pdf*)


THE INFORMATION CONTAINED IN THIS DRAWING IS THE SOLE PROPERTY OF UNIT ELECTRONICS. ANY REPRODUCTION IN PART OR WHOLE IS STRICTLY PROHIBITED WITHOUT WRITTEN PERMISSION.
PROPRIETARY
QWIIC
Touch Sensor
Mounting Holes
H1
H2
H3
H4
Level Shifter
Dip Switch
Electrodes/LEDs
Voltage Regulator
Pin Headers
Power On LED
MPR121QR2
AP2112K_3.3V
Title: UE0116 - QWIIC Touch Sensor v4
SKU: UE0116
REV: 1